

中山大学计算机学院

人工智能

本科生实验报告

(2022 学年春季学期)

课程名称: Artificial Intelligence

教学班级	20240574	专业 (方向)	计算机科学与技术
学号	23320093	姓名	林宏宇

一、实验题目

深度学习——中药图片分类任务

利用pytorch 框架搭建神经网络实现中药图片分类，其中中药图片数据分为训练集 train 和测试集 test，训练集仅用于网络训练阶段，测试集仅用于模型的性能测试阶段。

训练集和测试集均包含五种不同类型的中药图片：baihe、dangshen、gouqi、huaihua、jinyinhua。请合理设计神经网络架构，利用训练集完成网络训练，统计网络模型的训练准确率和测试准确率，画出模型的训练过程的 loss 曲线、准确率曲线。

提示：

1. 最后提交的代码只需包含性能最好的实现方法和参数设置。只需提交一个代码文件，请不要提交其他文件。
2. 本次作业可以使用 pytorch 库、numpy 库、matplotlib 库以及 python 标准库。
3. 数据集可以在 Github 上下载。
4. 模型的训练性能以及测试性能均作为本次作业的评分标准。
5. 测试集不可用于模型训练。
6. 不能使用开源的预训练模型进行训练。

二、实验内容

1. 算法原理

卷积神经网络 (CNN) 是一种专门用于处理具有类似网格结构的数据 (如本次实验中的图像) 的深度学习模型。CNN 主要由卷积层 (Conv2d)、激活函数 (如 ReLU)、池化层 (MaxPool2d) 和全连接层 (Linear) 组成。

核心原理:

卷积层: 通过卷积核在输入图片上滑动, 提取局部特征 (如边缘、纹理等), 并生成特征图。

激活函数: 如 ReLU 函数, 增加网络的非线性表达能力。

池化层: 如最大池化 (MaxPool2d), 降低特征图的空间尺寸, 减少参数量和计算量, 同时保留主要特征。

全连接层: 将卷积和池化提取到的高维特征映射到具体的分类输出空间, 实现最终的分类。

2. 原理应用

CNN 结构:

1. 输入: 3 通道彩色图片 (图像大小设置为 224×224)
2. 两层卷积+池化: 逐步提取图片的空间特征, 并降低空间维度
3. 自动计算展平后的特征维度: 保证全连接层输入维度正确
4. 全连接层: 输出为 5 类 (本题有 5 个中药类别)

CNN 的应用流程:

1. 数据预处理: 将图片统一缩放到 224×224 , 归一化处理。
2. 训练集加载: 用 ImageFolder 自动将图片文件夹结构映射为标签, 便于监督学习。
3. 模型训练: 通过多轮 (epoch) 遍历训练集, CNN 自动学习图片的空间特征, 优化参数以最小化交叉熵损失。
4. 测试与评估: 用测试集图片进行前向推理, 输出每张图片的预测类别, 与真实标签对比, 计算准确率。
5. 可视化: 绘制训练过程中的损失和准确率曲线, 便于分析模型收敛情况。

3. 伪代码

定义数据预处理 transform (缩放、转张量、归一化)

定义 CNN 模型结构

- 两层卷积+池化
- 自动计算展平维度
- 全连接层输出类别数

加载训练集和测试集 (ImageFolder)

创建 DataLoader (批量加载数据)

初始化 CNN 模型、优化器、损失函数

for epoch in 训练轮数:

 for 每个训练批次(b_x, b_y):

 前向传播: `output = cnn(b_x)`



计算损失: `loss = loss_func(output, b_y)`

反向传播与优化

记录 `loss` 和 `accuracy`

绘制 `loss` 和 `accuracy` 曲线

for 每个测试样本(`t_x, t_y`):

 前向传播: `test_output = cnn(t_x)`

 预测类别: `pred_y = argmax(test_output)`

 输出预测标签名和真实标签名

 统计准确率

输出测试集准确率

4. 关键代码展示（带注释）

1. CNN 结构

```
class CNN(nn.Module):
    def __init__(self):
        super(CNN, self).__init__()

        self.conv1 = nn.Sequential(          # 输入 224*224
            nn.Conv2d(
                in_channels=3,                # 彩色图片 三通道 即 3*224*224
                out_channels=16,              # 输出维度
                kernel_size=5,                # 过滤器 5*5
                stride=1,                     # 步长
                padding=0,                    # 四周填充
            ),                                # 输出 16*220*220
            nn.ReLU(),                        # 激活函数
            nn.MaxPool2d(kernel_size=2),      # 2*2 区域取最大值->输出 16*110*110
        )

        self.conv2 = nn.Sequential(          # 输入 16*110*110
            nn.Conv2d(16, 32, 5, 1, 2),      # 输出 32*110*110
            nn.ReLU(),                        # 激活函数
            nn.MaxPool2d(2),                  # 输出 32*55*55
        )

        # 自动计算展平后的特征维度（原视频代码为指定维度）
        self._initialize_fc()

    def _initialize_fc(self):                # 自动计算展平后的特征维度
        with torch.no_grad():
            x = torch.zeros(1, 3, 224, 224) # 虚拟输入
            x = self.conv1(x)
            x = self.conv2(x)
```



```
n_features = x.view(1, -1).shape[1] # 展平后的特征维度  
self.out = nn.Linear(n_features, 5) # 输出 5 类
```

```
def forward(self, x):  
    x = self.conv1(x) # 第一层  
    x = self.conv2(x) # 第二层  
    x = x.view(x.size(0), -1) # 展平  
    output = self.out(x) # 全连接层  
    return output, x
```

2. 数据加载和预处理

```
# 定义数据预处理的转换器  
transform = transforms.Compose([  
    transforms.Resize((224, 224)), # 调整图像大小  
    transforms.ToTensor(), # 将图像转换为张量  
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]) # 标准化  
)
```

```
# 加载训练数据集  
train_dataset = datasets.ImageFolder(root='train', transform=transform)  
train_loader = DataLoader(train_dataset, batch_size=8, shuffle=True, num_workers=4)  
  
# 加载测试数据集  
test_dataset = datasets.ImageFolder(root='test', transform=transform)  
test_loader = DataLoader(test_dataset, batch_size=1, shuffle=True, num_workers=1)
```

3. 训练

```
# 训练  
cnn = CNN() # 实例化 CNN 模型  
optimizer = torch.optim.Adam(cnn.parameters(), lr=0.0001) # 定义 Adam 优化器，设置学习率为 0.0001  
loss_func = nn.CrossEntropyLoss() # 定义交叉熵损失函数（适用于多分类任务）  
loss_list = [] # 记录损失值  
accuracy_list = [] # 记录准确率  
for epoch in range(10): # 10 个 epoch  
    for step, (b_x, b_y) in enumerate(train_loader): # 遍历训练集的每个 batch  
        output = cnn(b_x)[0] # 前向传播，获得模型输出  
        loss = loss_func(output, b_y) # 计算当前 batch 的损失  
        optimizer.zero_grad() # 梯度清零  
        loss.backward() # 反向传播，计算梯度  
        optimizer.step() # 更新模型参数  
    test_output, last_layer = cnn(b_x) # 再次前向传播，获取输出和特征  
    pred_y = torch.max(test_output, 1)[1].data.numpy() # 获取预测类别  
    accuracy = float((pred_y == b_y.data.numpy()).astype(int).sum()) / float(b_y.size(0)) # 计算准确率  
  
    loss_list.append(loss.data.numpy()) # 记录损失  
    accuracy_list.append(accuracy) # 记录准确率
```

4. 测试

```
# 测试
for step, (t_x, t_y) in enumerate(test_loader):
    test_output, _ = cnn(t_x)
    pred_y = torch.max(test_output, 1)[1].data.numpy()
    # 输出预测的标签名
    pred_names = [label_names[i] for i in pred_y]
    real_names = [label_names[i] for i in t_y.numpy()]
    print("第", step+1, "张图片测试: ")
    print('prediction:', pred_names)
    print('real:', real_names)
    if pred_names == real_names:
        count+=1
print("测试集准确率" ,count/len(test_loader))
```

5. 创新点&优化（如果有）

1. 数据增强

随机旋转、翻转、裁剪、颜色抖动等，提升模型泛化能力。

```
transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(15),
    transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2, hue=0.1),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])
```

2. 调整参数

2.1 学习率

影响模型收敛速度和最终效果。可以尝试更小或更大的学习率，如 0.01、0.0005 等。

2.2 优化器类型

目前用的是 Adam，可以尝试 SGD、SGD+Momentum、AdamW 等。

2.3 批量大小

影响训练稳定性和速度。可以尝试 16、64、128 等不同大小。

2.4 卷积层参数

卷积核数量、卷积核大小、步长、填充等。

可以增加卷积层数或每层的通道数。

2.5 池化方式和参数

可以尝试不同的池化核大小或类型。

2.6 训练轮数 epoch

增加训练轮数，观察模型是否继续提升。

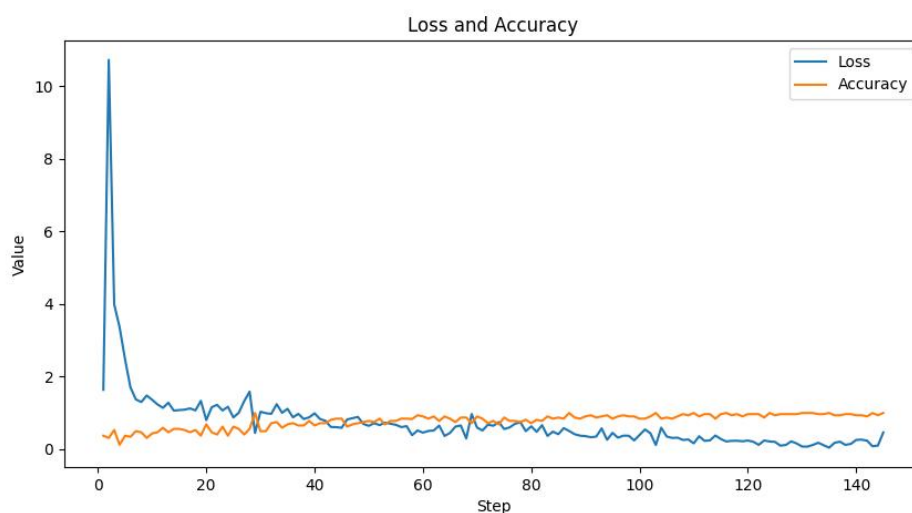
3. 使用更深层次的 CNN 结构

当前模型使用的是两层结构，可以考虑使用三层结构等，提高 CNN 结构的性能。

三、 实验结果及分析

1. 实验结果展示示例（可图可表可文字，尽量可视化）

训练结果：



Loss 损失值曲线和 Accuracy 准确率曲线结果如图

Loss 整体呈现波状递减，越来越接近于 0

Accuracy 整体呈现波状递增，最终趋近于 1

测试结果：

```
第 1 张图片测试：
prediction: ['dangshen']
real: ['dangshen']
第 2 张图片测试：
prediction: ['huaihua']
real: ['huaihua']
第 3 张图片测试：
prediction: ['jinyinhua']
real: ['jinyinhua']
第 4 张图片测试：
prediction: ['jinyinhua']
real: ['jinyinhua']
第 5 张图片测试：
prediction: ['huaihua']
real: ['huaihua']
第 6 张图片测试：
prediction: ['gouqi']
real: ['gouqi']
```

```
第 7 张图片测试:
prediction: ['baihe']
real: ['baihe']
第 8 张图片测试:
prediction: ['dangshen']
real: ['dangshen']
第 9 张图片测试:
prediction: ['baihe']
real: ['baihe']
第 10 张图片测试:
prediction: ['gouqi']
real: ['gouqi']
测试集准确率 1.0
```

2. 评测指标展示及分析

1. 损失值 Loss 曲线

Loss 曲线整体呈现波动下降趋势，逐渐接近于 0。

这说明模型在训练过程中不断优化参数，拟合能力增强，模型对训练数据的预测越来越准确，损失越来越小，收敛效果良好。

2. 准确率 Accuracy 曲线

Accuracy 曲线整体呈现波动上升趋势，最终趋近于 1。

准确率的提升说明模型对训练集的分类能力不断增强，最终达到较高的正确率，表明模型已经很好地学习到了训练数据的特征。

3. 测试集逐张预测结果

每张测试图片的预测标签与真实标签完全一致。

模型对测试集的泛化能力较强，能够准确识别每一类中药图片，说明模型没有出现明显的过拟合或欠拟合现象。

4. 测试集总体准确率

测试集准确率为 1.0（100%）。

模型在测试集上的表现非常优秀，所有图片均被正确分类。

这说明：

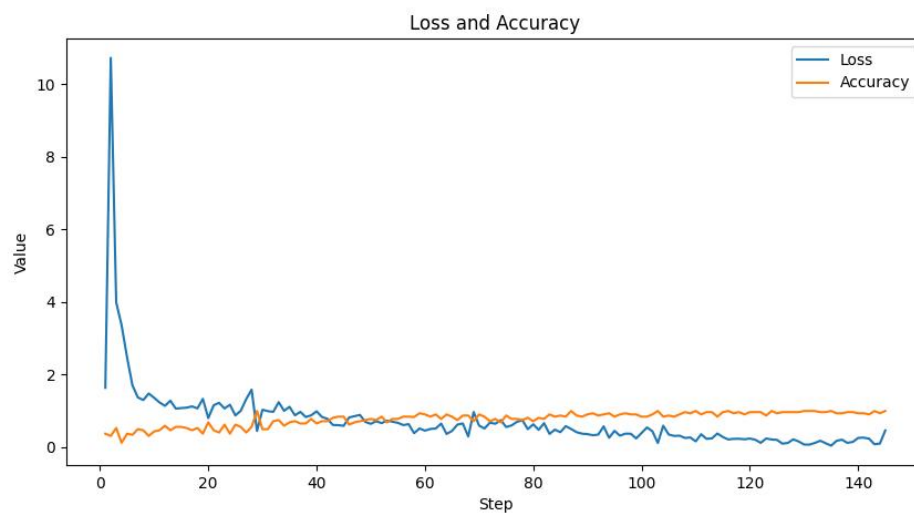
训练集与测试集分布一致，模型能够很好地泛化到未见过的数据。

数据集规模较小，类别区分明显，模型易于学习和区分。

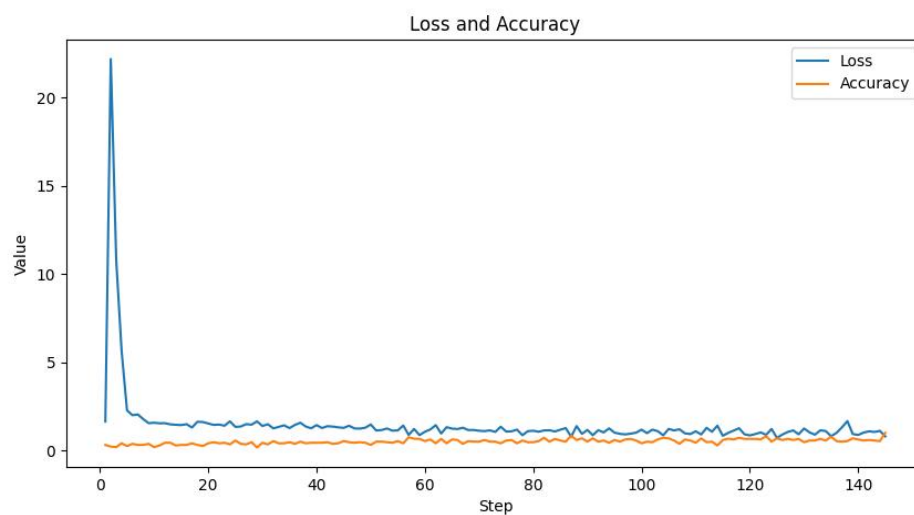
但是因为测试集过小，准确率高可能存在偶然性。

3. 优化一、数据增强

优化前：



优化后：



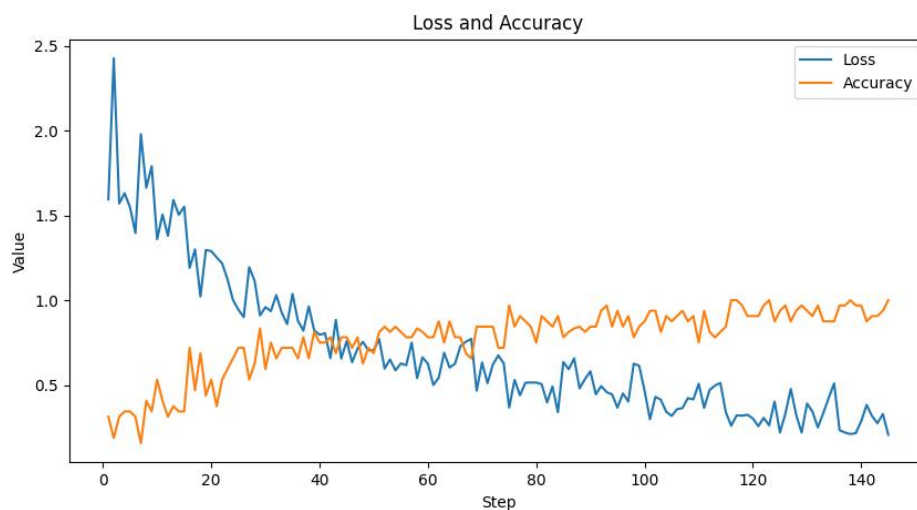
对比优化前的结果，训练效果反而变差了
数据增强操作在此次实验中不适用。

4. 优化二、调整参数

学习率、优化器类型、批量大小、训练轮数

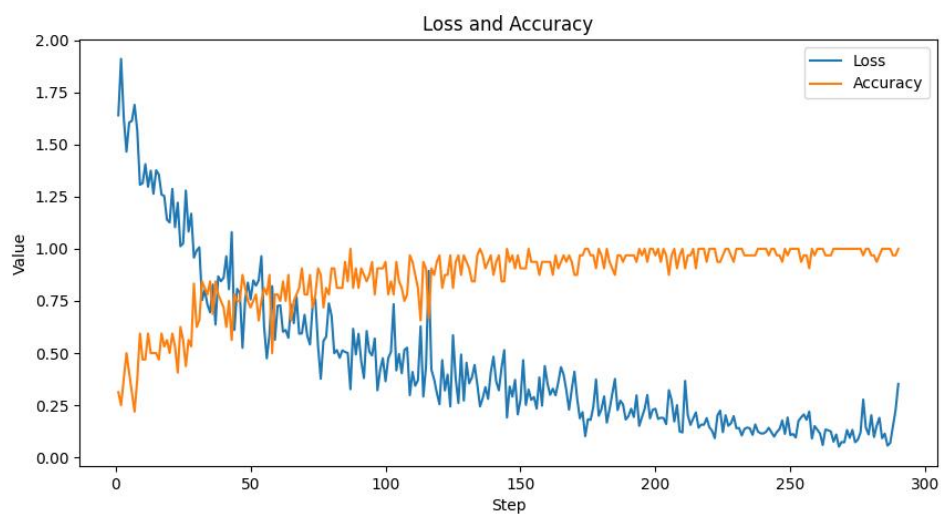
首先，将学习率设置为 **0.0001**

优化后：



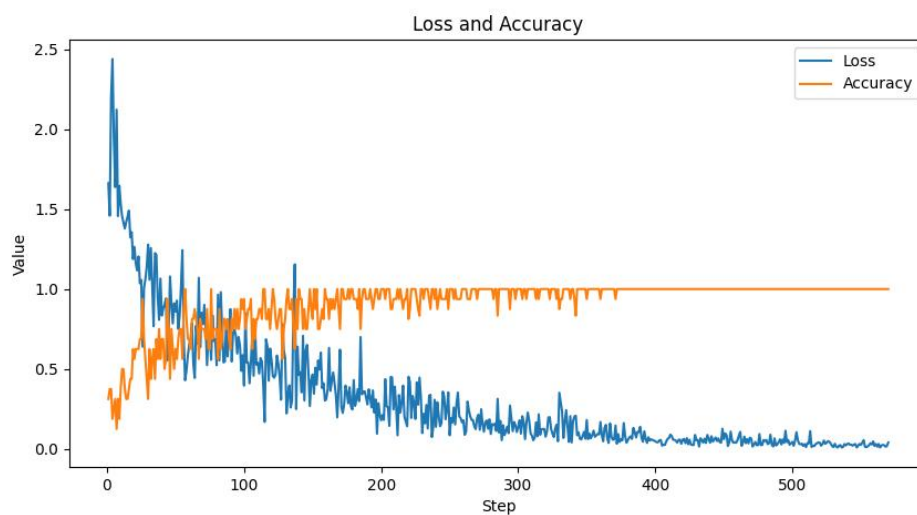
接着，将训练轮数设置为 **10**

优化后：



然后，将批量大小设置为 16
优化后：

训练结果：



测试结果：

```
第 1 张图片测试：
prediction: ['dangshen']
real: ['dangshen']
第 2 张图片测试：
prediction: ['huaihua']
real: ['huaihua']
第 3 张图片测试：
prediction: ['jinyinhua']
real: ['jinyinhua']
第 4 张图片测试：
prediction: ['jinyinhua']
real: ['jinyinhua']
第 5 张图片测试：
prediction: ['huaihua']
real: ['huaihua']
第 6 张图片测试：
prediction: ['gouqi']
real: ['gouqi']
第 7 张图片测试：
prediction: ['baihe']
real: ['baihe']
第 8 张图片测试：
prediction: ['dangshen']
real: ['dangshen']
第 9 张图片测试：
```

```
prediction: ['baihe']
```

```
real: ['baihe']
```

```
第 10 张图片测试:
```

```
prediction: ['gouqi']
```

```
real: ['gouqi']
```

```
测试集准确率 1.0
```

可知，经过参数调整后，训练效果和测试效果得到显著提升

5. 优化三、CNN 结构

当 CNN 结构层次增加时，相应的训练时间也会增加

因此在这里考虑两层的 CNN 结构

同时兼顾训练质量和训练时间

6. 总结

通过优化 CNN 模型、调整数据处理、调整参数等优化方法，模型的训练过程更加平稳，损失下降更快，准确率提升更明显。最终模型在测试集上依然保持 100% 的准确率，说明参数优化有效提升了模型的收敛速度和稳定性。

四、 思考题

1. 数据集规模与模型泛化能力的关系

本实验中，模型在测试集上达到了 100% 的准确率，说明模型对当前数据集的学习效果非常好。但由于数据集规模较小，类别区分明显，模型容易学习到训练集和测试集的特征，导致高准确率。实际应用中，若数据集更大且分布更复杂，模型的泛化能力可能会下降。因此，扩大数据集规模和多样性对于提升模型在实际场景下的表现非常重要。

2. 数据增强的适用性与局限性

实验中尝试了数据增强（如旋转、翻转、颜色扰动等），但发现训练效果反而变差。这说明数据增强并非在所有任务和数据集上都能提升效果。对于本实验的小样本、类别区分明显的中药图片，过度的数据增强可能引入了不必要的噪声，反而影响了模型的学习。因此，数据增强策略应根据具体数据特点灵活调整。

3. 模型结构与训练效率的权衡

实验中采用了两层卷积结构，兼顾了模型的表达能力和训练效率。虽然更深的网络理论上能提取更复杂的特征，但在小数据集和简单分类任务下，过深的网络容易导致过拟合且训练时间增加。因此，模型结构的设计应结合任务复杂度、数据规模 and 实际算力，保证实际可行性。



五、 参考资料

1. <https://blog.csdn.net/IronmanJay/article/details/128689946>
2. https://blog.csdn.net/v_JULY_v/article/details/51812459
3. https://blog.csdn.net/m0_63260018/article/details/132265213